BỘ CÔNG THƯƠNG · TRƯỜNG ĐẠI HỌC SAO ĐỎ · KHOA CÔNG NGHỆ THÔNG TIN

GIÁO TRÌNH HỌC PHẦN NỘI BỘ

Giáo trình Hệ sinh thái HDFS, MapReduce & Xử lý phân tán Apache Spark

Học phần: Dữ liệu lớn - Big Data (0101103389) | **Số tín chỉ:** 3 TC
Giảng viên phụ trách: ThS. Lê Thị Hồng | **Phòng học:** B1-302 / B1-204 | **Học kỳ:** HK2 2025-2026

Tóm tắt học liệu: Giáo trình phân tích kiến trúc đọc/ghi HDFS chịu lỗi, luồng MapReduce, cơ chế Lazy Evaluation, DAG Stage và bộ tối ưu Catalyst trong Apache Spark.

CHƯƠNG 1. HỆ THỐNG TỆP PHÂN TÁN HADOOP HDFS

- Kiến trúc Master/Worker: NameNode quản lý siêu dữ liệu không gian tên (cây thư mục, bảng ánh xạ tệp -> danh sách khối trong RAM + EditLog/FsImage); các DataNode lưu trữ trực tiếp các khối dữ liệu vật lý và gửi nhịp tim (Heartbeat) mỗi 3 giây.

- Kích thước khối mặc định (Block Size): 128 MB giúp giảm tỷ lệ thời gian tìm kiếm đầu đọc (seek time) so với thời gian truyền tuần tự. Ví dụ: một tệp nhật ký 500 MB được cắt thành 4 khối (3 khối 128 MB và 1 khối cuối 116 MB, không chiếm nguyên 128 MB đĩa cứng).

- Chiến lược đặt bản sao chịu lỗi (Rack-aware Replication = 3): Bản sao 1 đặt trên nút cục bộ (hoặc ngẫu nhiên), bản sao 2 đặt trên một tủ rack khác, bản sao 3 đặt trên một nút khác cùng tủ rack thứ hai -> chịu được lỗi hỏng toàn bộ một rack mà vẫn tối ưu băng thông ghi.

CHƯƠNG 2. MÔ HÌNH MAPREDUCE VÀ HẠN CHẾ I/O ĐĨA CỨNG

- Pha Map đọc từng phân mảnh (InputSplit), phát ra cặp trung gian (k1, v1) -> list(k2, v2). Combiner hoạt động như bộ "mini-reducer" cục bộ ngay tại nút Map để giảm lưu lượng mạng trước khi Shuffle (chỉ áp dụng cho phép toán có tính giao hoán và kết hợp như SUM, COUNT, MAX; không áp dụng trực tiếp cho trung bình cộng nếu không tách tổng và số đếm).

- Hạn chế: Giữa hai công việc MapReduce liên tiếp (ví dụ thuật toán lặp PageRank hay K-Means), kết quả trung gian bắt buộc phải ghi xuống HDFS rồi đọc lại từ đĩa, gây nghẽn I/O nghiêm trọng.

CHƯƠNG 3. KIẾN TRÚC APACHE SPARK & TỐI ƯU CATALYST

- Đánh giá lười (Lazy Evaluation): Các phép biến đổi (Transformations như map, filter, join, groupBy) chỉ ghi nhận vào đồ thị có hướng không chu trình (DAG). Spark chỉ thực sự khởi chạy công việc khi gặp một Hành động (Action như count, collect, write).

- Phân ranh giới Stage bằng Narrow vs Wide Transformation:

+ Biến đổi hẹp (Narrow: map, filter, union): Mỗi phân vùng cha chỉ đóng góp vào tối đa 1 phân vùng con -> Gộp chung vào một Stage và thực thi pipeline ngay trong bộ nhớ RAM mà không cần truyền mạng.

+ Biến đổi rộng (Wide: groupByKey, reduceByKey, join, distinct, repartition): Dữ liệu cùng khóa nằm rải rác trên nhiều nút phải trao đổi qua mạng (Shuffle) -> Tạo ranh giới tách Stage mới!

- DataFrame & Spark SQL vượt trội hơn RDD thuần nhờ bộ tối ưu truy vấn Catalyst (đẩy điều kiện lọc xuống sớm - Predicate Pushdown, cắt tỉa cột thừa - Column Pruning trên tệp Parquet) và bộ quản lý bộ nhớ nhị phân ngoài heap Tungsten.